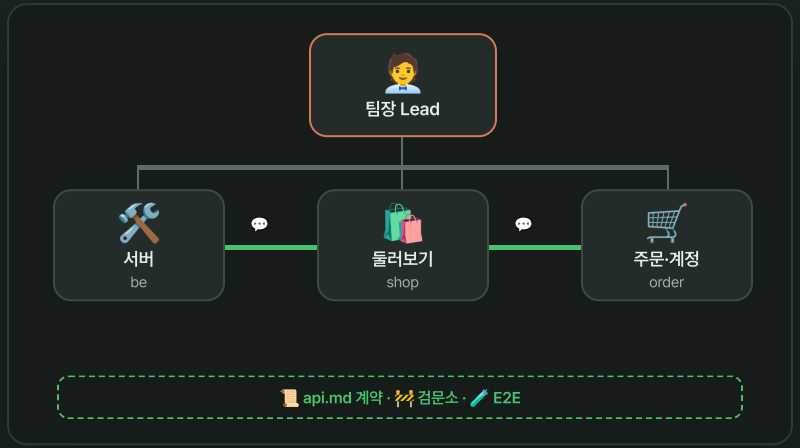
실습 자료 · 8시간

클로드 팀으로 쇼핑몰 처음부터 만들기

8시간 실습 · 빈 폴더에서 하루마켓까지 · 실습 코드와 강사 해설

Lead Teammates Task list



목차

들어가며	이 자료 구성	3
블록 0	오리엔테이션 — 빈 폴더에서 시작 (20분)	4
블록 1	설계 팀 — 계약서부터 (40분)	5
블록 2	계약 확정 — 내가 읽고 고치고 못 박기 (30분)	6
블록 3	뼈대 — 팀장 혼자 만든다 (40분)	7
블록 4	구현 팀 — 파일을 나눠 동시에 (90분)	8
블록 5	QA 팀원 — 사용자처럼 끝까지 눌러 보는 테스트 (40분)	9
블록 6	리뷰 팀 — 보안·성능·접근성 털기 (40분)	10
블록 7	수리 + 다듬기 팀 — "개쩌는" 마무리 (50분)	11
블록 8	최종 검증 — 처음 받는 사람처럼 (40분)	12
블록 9	발표 & 회고 (30분)	13

들어가며

이 자료 구성

빈 폴더에서 시작해 무신사·쿠팡 같은 **진짜 쇼핑몰 수준의 사이트**(가상 브랜드 **하루마켓**)를 클로드 팀으로 만듭니다. 블록마다 **목표**(왜 하는지) → 단계마다 **실습 코드**(⌨️ 터미널 / 💬 클로드 대화창, 줄 끝 `#` 은 설명) → 🧑‍🏫 **강사 해설** 순서입니다.

블록 0

오리엔테이션 — 빈 폴더에서 시작 (20분)

목표 — 오늘은 고칠 앱이 없습니다. 빈 폴더 하나에서 시작해, 하루가 끝나면 상품 목록·검색·상세·장바구니·주문·마이페이지가 모두 돌아가는 쇼핑몰이 남습니다. "팀에게 한 방에 시키면 망가지고, 순서를 지키면 버그 없이 나온다"를 직접 겪는 것이 오늘의 핵심입니다.

0-1. 프로젝트 폴더 만들기 (10분)

⌨️ **터미널**

```
node --version                                  # v20 이상이면 OK (오늘 쓰는 Vite·Playwright 가 20 이상 필요)
claude --version                                # Claude Code 설치 확인. 오래됐으면 claude update
mkdir -p ~/projects/haru-market && cd ~/projects/haru-market   # 빈 프로젝트 폴더 만들고 들어가기
git init                                        # 이 폴더를 git 저장소로 → 오늘 모든 단계를 커밋으로 남김
mkdir -p .claude docs                           # 클로드 설정 폴더와 설계 문서 폴더
```

💬 **클로드 대화창** (⌨️ `claude` 로 시작한 뒤)

```
.claude/settings.json 을 만들어줘. env 에 CLAUDE_CODE_EXPERIMENTAL_AGENT_TEAMS=1,   # 팀 기능 켜기 (이 프로젝트에서만)
permissions.allow 에 "Bash(npm run *)", "Bash(npm test*)", "Bash(npx playwright test*)", "Bash(git status*)", "Bash(git diff*)", "Bash(git log*)" 추가.   # 안전한 명령은 묻지 않고 실행 → 허락 요청 폭탄 방지
저장 후 파일 전체를 보여줘.   # 눈으로 확인
```

⌨️ **터미널**

```
/exit                                           # (💬 대화창에서) 새 설정을 읽게 클로드 끄기
claude                                          # 다시 켜기 → 이제 팀 기능이 켜진 상태
```

> 🧑‍🏫 **강사 해설**
>
> `.claude/settings.json` 에 `env` 와 `permissions.allow` 두 칸이 들어가면 정답입니다. 블록 0 에서 미리 허락 목록을 넣는 이유는, 오늘은 팀원 3~4명이 각자 `npm test`, `npm run build` 를 수십 번 돌리기 때문입니다. 허락 목록이 없으면 그 요청이 전부 팀장 화면으로 올라와서 학생이 계속 Enter 만 누르게 됩니다. `git push`, `rm` 같은 되돌리기 어려운 명령은 일부러 넣지 않았습니다.
>
> 설정을 넣은 뒤 반드시 껐다 켜야 합니다. 클로드는 켜질 때 설정을 읽기 때문에, 이걸 빼먹으면 블록 1 에서 팀 대신 심부름꾼이 생깁니다.

0-2. 오늘 만들 것 보여주기 (10분)

⌨️ **화면에 띄우고 같이 읽기** (실제로 치는 명령 아님)

```
하루마켓 = 가상 쇼핑몰 브랜드 (실제 회사 이름·로고 사용 금지)              # 저작권·상표 문제 없게 가상 브랜드
홈: 배너 슬라이드, 타임특가(카운트다운), 카테고리 바로가기, 랭킹 TOP 10      # 쿠팡·지마켓 첫 화면 느낌
목록: 카테고리별 상품, 정렬(인기·낮은가격·높은가격·할인율), 가격 필터, 페이지 나누기   # 무신사 목록 느낌
검색: 검색창 자동완성, 결과 없을 때 안내                                  # 실제 쇼핑몰 필수 기능
상세: 이미지, 할인가, 별점·리뷰, 옵션(사이즈/색상), 수량, 장바구니·바로구매      # 상품 상세
장바구니 → 주문(가짜 결제) → 주문 완료 → 마이페이지 주문내역                # 돈이 오가지 않는 가짜 결제
회원: 가입·로그인·로그아웃, 찜(위시리스트), 최근 본 상품                   # 로그인해야 쓸 수 있는 기능
모바일 화면에서도 깨지지 않기                                             # 반응형
```

> 🧑‍🏫 **강사 해설**
>
> 학생들에게 먼저 "이걸 클로드에게 한 문장으로 '무신사 같은 쇼핑몰 만들어줘' 하면 어떻게 될까요?"라고 물어보세요. 답은 "뭔가 나오긴 하는데, 장바구니 숫자가 안 맞거나, 로그인하면 화면이 깨지거나, 다시 켜면 데이터가 사라지는" 결과입니다. 팀으로 한 방에 시키면 더 나빠집니다. 팀원마다 상품 데이터 모양을 다르게 상상하고 만들기 때문에, 합치는 순간 화면과 서버가 서로 말이 안 통합니다.
>
> 그래서 오늘은 순서를 지킵니다. **설계 팀이 계약서를 쓰고(블록 1) → 내가 확정하고(블록 2) → 팀장 혼자 뼈대를 만들고(블록 3) → 그 위에서 구현 팀이 파일을 나눠 만들고(블록 4) → QA 팀원이 사용자처럼 끝까지 눌러 보고(블록 5) → 리뷰 팀이 털고(블록 6) → 고치고 다듬고(블록 7) → 처음 받는 사람처럼 검증합니다(블록 8).** 이 순서 자체가 "버그 없이 만드는 방법"이고, 오늘 배우는 내용입니다.
>
> 실제 회사 이름과 로고를 쓰지 않는 이유도 짚어 주세요. 무신사·쿠팡의 로고나 상호를 그대로 쓰면 상표 문제가 생기고, 포트폴리오에도 쓸 수 없습니다. "느낌은 참고하되 브랜드는 내 것"이 원칙입니다. 상품 이미지는 외부 사진 대신 **그라데이션 + 이모지**로 만들어서 저작권과 인터넷 연결 문제를 동시에 피합니다.

블록 1

설계 팀 — 계약서부터 (40분)

목표 — 코드를 한 줄도 쓰기 전에 팀으로 설계 문서를 만듭니다. 팀원들이 나중에 각자 다른 파일을 만들어도 합쳐지는 건 **모두가 같은 계약서(데이터 모양, API 주소, 화면 목록)를 보고 만들기 때문**입니다. 기획은 파일을 고치지 않아서 팀이 가장 안전하게 잘하는 일이기도 합니다.

1-1. 설계 팀 꾸리기 (25분)

💬 **클로드 대화창**

```
"하루마켓"이라는 가상 쇼핑몰을 처음부터 만들 거야. 무신사·쿠팡 같은 수준이 목표고, 실제 브랜드 이름·로고는 쓰지 않아.  # 팀원은 이 대화를 모르니 배경을 직접 적기
기능: 홈(배너 슬라이드, 타임특가 카운트다운, 카테고리, 랭킹), 카테고리 목록(정렬·가격 필터·페이지), 검색(자동완성),  # 0-2 의 기능 목록 그대로
상품 상세(옵션·수량·별점·리뷰), 장바구니, 주문(가짜 결제), 주문 완료, 마이페이지(주문내역·찜·최근 본 상품), 회원가입·로그인. 모바일 대응.
기술: 프론트 React + Vite + react-router, 백엔드 Node + Express, 데이터는 서버 메모리 + JSON 파일 저장, 이미지는 그라데이션+이모지.  # 기술을 미리 못 박아야 팀원끼리 안 갈림
agent team 으로 설계 팀 4명을 뽑아줘. 모델은 모두 Sonnet. 코드는 쓰지 마.  # 이번 블록은 문서만
- pm: 화면 목록과 화면별 기능, 사용자 흐름(검색→상세→장바구니→주문→마이페이지)을 docs/spec.md 에  # 무엇을 만드나
- arch: 데이터 모양(상품·옵션·회원·장바구니·주문)과 API 계약(주소, 요청, 응답 JSON 예시, 에러 코드)을 docs/api.md 에  # 화면과 서버가 지킬 약속
- designer: 색·글꼴·간격·버튼·카드 규칙과 공통 컴포넌트 목록(Header, ProductCard 등)을 docs/design.md 에  # 화면이 한 사이트처럼 보이게
- critic: 세 문서를 읽고 빠진 경우(품절, 재고 부족, 비로그인, 새로고침, 빈 장바구니, 가격 계산)를 찾아서 해당 팀원에게 직접 반박  # 악마의 변호인
서로 문서를 읽고 메시지로 맞춰. 끝나면 세 문서가 서로 어긋나는 곳이 없는지 네가 확인하고 요약해줘.  # 팀장의 최종 대조
```

⌨️ **팀이 일하는 동안**

```
↓ (입력창을 비운 채) → critic 고르고 Enter  # critic 화면 열기 (↑ 는 이전 입력이 나옴)
가격 계산에서 실수가 날 만한 곳만 골라서 arch 에게 다시 물어봐  # critic 에게 직접 지시
Esc                                        # 팀장 화면으로 (팀원 작업이 멈추면 다시 들어가 "계속해")
Ctrl + T                                   # 할 일 보드 보기
```

> 🧑‍🏫 **강사 해설**
>
> 결과로 `docs/spec.md`, `docs/api.md`, `docs/design.md` 세 파일이 생기면 정답입니다. 잘 나온 결과에는 이런 것들이 들어 있습니다.
>
> - **spec.md** — 화면 9~11개(홈, 카테고리, 검색, 상세, 장바구니, 주문서, 주문 완료, 로그인/가입, 마이페이지)와 각 화면의 기능, 비로그인일 때 어떻게 되는지.
> - **api.md** — `GET /api/products?category=&sort=&minPrice=&maxPrice=&page=`, `GET /api/products/:id`, `GET /api/search/suggest?q=`, `POST /api/auth/signup`, `POST /api/auth/login`, `GET/POST/PATCH/DELETE /api/cart`, `POST /api/orders`, `GET /api/orders`, `GET/POST/DELETE /api/wishlist` 같은 주소와, 각각의 요청·응답 JSON 예시, 에러 모양(`{ "error": { "code": "OUT_OF_STOCK", "message": "..." } }`).
> - **design.md** — 브랜드 색, 글꼴, 간격 단위(4px/8px), 카드 모양, 할인율 빨간 글씨, 버튼 종류, 공통 컴포넌트 목록.
>
> critic 의 반박에서 꼭 나와야 할 것은 **가격은 서버가 계산한다**는 원칙입니다. 화면이 계산한 가격을 그대로 주문에 쓰면, 사용자가 브라우저에서 값을 바꿔 1원에 주문할 수 있습니다. 그리고 **돈은 정수(원 단위)로만** 다룬다는 원칙입니다. 할인율 계산에서 소수점이 생기면 장바구니 합계가 1원씩 어긋나는 버그가 생깁니다. 이 두 가지가 api.md 에 들어가면 critic 이 제 몫을 한 것입니다.
>
> 팀으로 설계하는 이유는 관점이 서로 독립적이기 때문입니다. 혼자 설계시키면 화면 목록은 잘 나오지만 "품절 상품을 장바구니에 담아 둔 상태에서 주문하면?" 같은 경우를 잘 놓칩니다. critic 이 다른 팀원에게 직접 따지면서 이런 구멍이 문서 단계에서 막힙니다. 코드를 쓰기 전에 막는 버그가 가장 싼 버그입니다.

1-2. 설계 팀 퇴근 + 저장 (5분)

💬 **클로드 대화창**

```
팀원 모두 퇴근시켜줘.  # 끝난 팀원 종료. 살아 있으면 말을 걸 때마다 깨어나 사용량을 씀
```

⌨️ **터미널** (새 터미널 창에서)

```
cd ~/projects/haru-market
git add . && git commit -m "docs: 하루마켓 설계 (spec, api, design)"  # 설계 문서 세이브 포인트
```

> 🧑‍🏫 **강사 해설**
>
> 여기서 커밋하는 이유는, 블록 2 에서 학생이 문서를 직접 고치기 때문입니다. 커밋을 해 두면 `git diff` 로 "팀이 쓴 것"과 "내가 고친 것"이 구분되어 보입니다. 팀원을 바로 퇴근시키는 이유는, 설계 팀은 이제 할 일이 없는데 살아 있으면 팀장이 진행 상황을 물을 때마다 깨어나서 자기 대화 전체를 다시 읽으며 사용량을 쓰기 때문입니다.

블록 2

계약 확정 — 내가 읽고 고치고 못 박기 (30분)

목표 — 팀이 쓴 설계를 내가 직접 읽고 확정합니다. 그리고 앞으로 모든 팀원이 자동으로 읽는 규칙 파일 `CLAUDE.md` 를 만듭니다. 팀원은 팀장과 나눈 대화를 모르지만 `CLAUDE.md` 는 켜질 때 자동으로 읽기 때문에, 여기에 적은 규칙은 모든 팀원이 지킵니다.

2-1. 설계 검토 (15분)

💬 클로드 대화창

```
docs/spec.md, docs/api.md, docs/design.md 를 읽고, 서로 어긋나는 곳과 애매한 곳을 표로 뽑아줘.   # 사람이 보기 전에 한 번 더 대조
특히 api.md 의 응답 JSON 필드 이름이 spec.md 의 화면에서 쓰는 이름과 같은지,   # 필드 이름 불일치 = 합칠 때 가장 흔한 버그
가격·할인·재고·주문 합계 계산 규칙이 한 곳에만 정의돼 있는지 확인해.   # 계산 규칙이 두 군데 있으면 언젠가 달라짐
```

→ 표를 보고 학생이 직접 결정해서 💬 `1번은 A 로, 2번은 B 로 정하고 문서에 반영해줘` 처럼 지시

👩‍🏫 강사 해설

보통 5~10개의 어긋남이 나옵니다. 자주 나오는 것은 이런 것들입니다. 화면에서는 `salePrice` 라고 부르는데 API 에서는 `discountedPrice`, 상품 id 가 어떤 곳은 숫자이고 어떤 곳은 문자열, 장바구니에 담는 단위가 "상품"인지 "상품+옵션 조합"인지 애매함, 품절 옵션을 화면에서 숨길지 회색으로 보여줄지 정해지지 않음.

이걸 **학생이 직접 정하게 하는 게** 이 단계의 핵심입니다. 클로드에게 "알아서 정해"라고 하면 정해지긴 하지만, 오후에 팀원이 만들다가 막혔을 때 학생이 "그거 왜 그렇게 했지?"를 모릅니다. 계약은 사람이 서명하는 것이라고 비유하면 됩니다.

2-2. CLAUDE.md — 모든 팀원이 읽는 규칙 (15분)

💬 클로드 대화창

```
프로젝트 루트에 CLAUDE.md 를 만들어줘. 모든 팀원이 자동으로 읽는 규칙이니까 짧고 분명하게.   # 팀원은 대화를 모르지만 CLAUDE.md 는 읽음
- 기술: client/ = React+Vite+react-router, server/ = Node+Express. 다른 라이브러리 추가는 팀장 허락 필요   # 팀원이 마음대로 라이브러리를 늘리면 충돌
- 계약: API 주소·필드 이름·에러 모양은 docs/api.md 가 정답. 바꿔야 하면 코드 말고 팀장에게 먼저 말하기   # 계약은 한 사람만 바꿈
- 돈: 모든 금액은 정수(원). 가격·할인·합계 계산은 server/src/pricing.js 한 곳에서만   # 계산 규칙 한 곳
- 폴더: client/src/pages, client/src/components, client/src/api.js(서버 호출 모음), server/src/routes, server/src/data   # 파일 위치 통일
- 명령: npm run dev(둘 다 실행), npm test(서버 테스트), npm run build(프론트 빌드), npm run e2e(브라우저 테스트)   # 모두 같은 명령 사용
- 완료 조건: npm test 와 npm run build 가 통과해야 완료. 자기 담당 파일만 git add 해서 "이름: 한 일" 로 커밋   # 검문소와 같은 규칙
- 금지: 실제 브랜드 이름·로고, 외부 이미지 URL, 카드번호 입력칸   # 상표·외부 의존·결제 정보 금지
```

⌨️ 터미널

```
git add . && git commit -m "docs: 계약 확정 + CLAUDE.md"   # 확정본 세이브
```

👩‍🏫 강사 해설

`CLAUDE.md` 가 오늘 실습에서 가장 중요한 파일입니다. Claude Code 공식 문서에 따르면 팀원은 켜질 때 `CLAUDE.md` 와 프로젝트 설정을 자동으로 읽지만, 팀장과 나눈 대화는 넘겨받지 않습니다. 그래서 "모든 팀원이 꼭 지켜야 하는 것"은 대화로 말하지 말고 `CLAUDE.md` 에 적어야 합니다.

"가격 계산은 `pricing.js` 한 곳에서만"이 들어간 이유를 설명해 주세요. 상품 카드, 상세, 장바구니, 주문서가 각자 할인가를 계산하면, 한 곳만 반올림 방식이 달라도 "상세에서는 29,900원인데 장바구니에서는 29,901원"이 됩니다. 쇼핑몰에서 가장 흔하고 가장 신뢰를 잃는 버그입니다. 계산을 한 곳에 모으면 이 버그는 구조적으로 생길 수 없습니다.

`CLAUDE.md` 는 짧아야 합니다. 길면 팀원마다 매번 읽느라 사용량이 늘고, 정작 중요한 규칙이 묻힙니다. 30줄 안쪽이 적당합니다.

블록 3

뼈대 — 팀장 혼자 만든다 (40분)

목표 — 팀원을 부르기 전에 **팀장 혼자** 프로젝트 뼈대(설정 파일, 폴더, 공통 부품, 가짜 데이터, 검문소)를 만듭니다. `package.json` 이나 라우터 설정처럼 모두가 건드리는 파일을 팀원 여럿이 동시에 만들면 반드시 덮어쓰기 사고가 나기 때문입니다. "팀 전에 혼자"가 이 블록의 교훈입니다.

3-1. 뼈대 만들기 (25분)

💬 **클로드 대화창**: **Shift + Tab** 으로 `plan mode` 로 바꾼 뒤

```
팀원 없이 너 혼자 프로젝트 뼈대를 만들 계획을 세워줘. CLAUDE.md 와 docs/ 를 따라.   # 공통 파일은 한 명이 만든다
- 루트 package.json: npm workspaces(client, server), 스크립트 dev(concurrently 로 둘 다), test, build, e2e   # 명령 통일
- server: Express 앱(app.js 와 index.js 분리, 테스트에서 app 만 불러 쓰게), /api/health, 에러 응답 공통 함수   # 테스트하기 좋은 구조
- server/src/pricing.js: 할인가·합계 계산 함수와 그 테스트   # 계산 규칙 한 곳 + 처음부터 테스트
- server/src/data/seed.js: 5개 카테고리 × 12개 = 상품 60개 가짜 데이터(옵션·재고·별점·리뷰 수·할인율·이모지·그라데이션 색 포함)   # 화면을 채울 데이터
- client: Vite React, 라우터에 spec.md 의 화면 전부를 "준비 중" 빈 페이지로 연결, Header·Footer·ProductCard 공통 컴포넌트, design.md 의 색·간격 CSS 변수   # 팀원은 빈 페이지를 채우기만
- client/src/api.js: api.md 의 모든 API 호출 함수 이름만 먼저(내용은 TODO)   # 화면 팀원이 같은 함수 이름을 쓰게
- vite 개발 서버의 /api 요청을 Express 로 넘기는 proxy 설정   # 화면과 서버 주소 맞추기
계획을 먼저 보여주고, 내가 OK 하면 만들어. 다 만든 뒤 npm install, npm test, npm run build 가 통과하는 것까지 확인해.   # 뼈대가 초록불인 상태로 시작
```

→ 계획을 읽고 💬 `OK` (Shift + Tab 으로 계획 모드를 끄고)

⌨️ **터미널** (새 터미널)

```
cd ~/projects/haru-market
npm test                                  # pricing 테스트 통과 확인
npm run build                             # 프론트 빌드 통과 확인
npm run dev                               # 브라우저에서 http://localhost:5173 → 헤더와 "준비 중" 페이지들이 보이면 OK. 끌 때 Ctrl + C
```

> 🧑‍🏫 **강사 해설**
>
> 결과로 `npm run dev` 를 켜면 하루마켓 헤더와 메뉴가 보이고, 메뉴를 누르면 "준비 중"이라고 적힌 빈 페이지들로 이동하면 정답입니다. `npm test` 에는 pricing 테스트 몇 개가 통과하고, `npm run build` 가 에러 없이 끝나야 합니다.
>
> 이 블록이 "버그 없이"의 절반입니다. 팀원들이 동시에 `package.json` 에 라이브러리를 추가하면 서로의 변경을 덮어써서 "어제 됐던 게 오늘 안 되는" 상태가 됩니다. 라우터 파일(`App.jsx`)도 마찬가지입니다. 화면 담당 둘이 각자 자기 페이지를 라우터에 추가하다가 한 명의 추가가 사라집니다. 이런 **공통 파일은 팀장 혼자 먼저 다 만들어 두고**, 팀원은 이미 연결된 빈 페이지를 채우기만 하게 하면 충돌할 일이 없습니다.
>
> `app.js` 와 `index.js` 를 나누는 이유는 테스트 때문입니다. 테스트에서 서버를 실제 포트로 띄우지 않고 `app` 만 불러서 요청을 보낼 수 있어야, 팀원들이 동시에 테스트를 돌려도 포트가 겹치지 않습니다.
>
> `api.js` 에 함수 이름만 먼저 만들어 두는 것도 중요합니다. 화면 팀원 두 명이 같은 API 를 각자 `getProduct`, `fetchProductDetail` 처럼 다른 이름으로 만들면 코드가 두 벌이 되고, 나중에 한쪽만 고쳐져서 버그가 됩니다.
>
> 학생 화면에서 `npm install` 이 실패하면 대부분 회사·학교 네트워크의 npm 차단이거나 Node 버전이 낮은 경우입니다. `node --version` 부터 확인하세요.

3-2. 검문소 설치 + 세이브 (15분)

💬 **클로드 대화창**

```
.claude/settings.json 에 hook 두 개를 추가해줘. 기존 env, permissions 는 유지.   # 사람 대신 자동 검사
- TaskCompleted: "npm test 1>&2 && npm run build 1>&2 ¦¦ exit 2"   # 팀원이 "완료" 하려는 순간 테스트+빌드. 실패하면 완료 거부
- TeammateIdle: "test -z \"$(git status --porcelain)\" ¦¦ (echo '커밋 안 한 변경이 있어. 자기 파일만 커밋하고 쉬어' 1>&2; exit 2)"   # 커밋 안 하고 쉬려 하면 막음
```

⌨️ **터미널**

```
git add . && git commit -m "chore: 프로젝트 뼈대 + 검문소"   # 팀이 망쳐도 여기로 돌아올 수 있는 세이브 포인트
git log --oneline                                          # 커밋 3개(설계, 계약, 뼈대)가 보이면 OK
```

> 🧑‍🏫 **강사 해설**
>
> `TaskCompleted` 는 팀원이 할 일을 "완료"로 표시하려는 순간 실행됩니다. `npm test` 와 `npm run build` 가 둘 다 통과해야 완료가 받아들여지고, 하나라도 실패하면 `exit 2` 로 완료가 거부되면서 실패 내용(`1>&2` 로 보낸 출력)이 팀원에게 전달됩니다. 팀원은 그걸 읽고 고친 뒤 다시 완료를 시도합니다. 종료 코드가 2 여야만 막힌다는 점을 꼭 짚어 주세요.
>
> `TeammateIdle` 은 팀원이 일을 멈추고 쉬려는 순간 실행됩니다. 커밋 안 한 파일이 있으면 쉬지 못하게 막아서, "다 했다면서 저장은 안 한" 상태를 없앱니다. 다만 다른 팀원이 작업 중인 파일까지 걸릴 수 있어서 가끔 엉뚱하게 막히기도 합니다. 그럴 때는 팀원이 "그건 내 파일이 아니다"라고 보고하므로, 강사가 상황을 보고 이 hook 만 빼도 됩니다.
>
> 검문소를 뼈대와 같은 블록에서 다는 이유는, 뼈대가 **초록불(테스트·빌드 통과) 상태**여야 검문소가 의미가 있기 때문입니다. 처음부터 빨간불이면 모든 완료가 거부되어 팀이 아무것도 못 합니다.

☕ 휴식 (10분)

블록 4

구현 팀 — 파일을 나눠 동시에 (90분)

목표 — 이제 팀원 3명이 동시에 쇼핑몰 기능을 채웁니다. 계약서(api.md), 규칙(CLAUDE.md), 뼈대, 검문소가 다 있으므로 팀원은 **자기 파일만** 채우면 되고, 합칠 때 말이 안 통하는 일이 없습니다. 실제로 팀이 가장 큰 속도를 내는 구간입니다.

4-1. 업무 지시서 먼저 (15분)

💬 **클로드 대화창**: **Shift + Tab** 으로 `plan mode`

```
docs/ 와 CLAUDE.md 를 바탕으로 구현 팀 계획을 세워줘. 팀원 3명, 모델은 모두 Sonnet.   # 3명 = 사용량과 속도의 균형
파일 소유표:   # 파일마다 주인 1명 → 덮어쓰기 사고 방지
- be: server/src/** (pricing.js 포함), server/test/**   # 서버 전부 + 서버 테스트
- shop: client/src/pages/Home, Category, Search, ProductDetail 와 client/src/components/shop/**   # 둘러보기 화면
- order: client/src/pages/Cart, Checkout, OrderComplete, Login, Signup, MyPage 와 client/src/components/order/**, client/src/store/**   # 사고 · 계정 화면
- 팀장(너): package.json, App.jsx(라우터), client/src/api.js, 공통 컴포넌트. 팀원이 바꿔 달라고 하면 네가 바꿈   # 공통 파일은 팀장만
할 일 보드는 팀원당 4~6개. 선행 작업: be 의 상품 API → shop 의 목록·상세 / be 의 인증·장바구니 API → order 의 화면.   # 순서가 있는 일은 잠가 두기
각 팀원 업무 지시서(역할/파일 범위/배경/할 일/완료 조건)를 먼저 보여주고, 내가 OK 하면 그때 팀원을 뽑아.   # 뽑기 전에 내가 확인
```

→ 지시서를 읽고 **파일 범위가 겹치지 않는지**, **api.md 를 따르라는 말이 들어 있는지** 확인 후 💬 `OK`

> 🧑‍🏫 **강사 해설**
>
> 지시서에서 학생이 꼭 확인할 것은 세 가지입니다. 첫째, 파일 범위가 겹치지 않는지(예: shop 과 order 가 둘 다 `Header` 를 고친다고 되어 있으면 안 됨). 둘째, 각 지시서에 "api.md 의 주소와 필드 이름을 그대로 따른다"가 들어 있는지. 셋째, 완료 조건에 "테스트와 빌드 통과"가 들어 있는지.
>
> 나누는 기준을 "프론트 / 백엔드 / 디자인"처럼 **기술**로 나누지 않고 "서버 / 둘러보기 화면 / 사고·계정 화면"처럼 **파일 묶음**으로 나눈 이유를 설명해 주세요. 디자인 담당을 따로 두면 그 팀원이 모든 화면 파일을 건드려야 해서 다른 팀원과 반드시 겹칩니다. 팀원 나누기의 기준은 언제나 "파일이 안 겹치게"입니다.
>
> 선행 작업을 거는 이유는, 상품 API 가 없는데 shop 이 목록 화면을 만들면 가짜 데이터로 만들었다가 나중에 다시 고쳐야 하기 때문입니다. 다만 shop 은 기다리는 동안 api.md 의 응답 예시로 화면 모양을 먼저 만들 수 있으므로 완전히 노는 건 아닙니다.

4-2. 팀 가동 (60분)

💬 **클로드 대화창**: **Shift + Tab** 으로 계획 모드를 끄고

```
OK. 팀원을 뽑아서 진행해줘. 각자 할 일 하나 끝날 때마다 npm test 와 npm run build 를 돌리고,   # 작게 만들고 바로 검사
자기 파일만 git add 해서 "팀원이름: 한 일" 로 커밋해. git add . 은 금지.   # 남의 작업 중간 상태가 섞이지 않게
api.md 와 다르게 만들어야 할 것 같으면 코드를 바꾸지 말고 너(팀장)에게 먼저 메시지로 물어봐.   # 계약 변경은 한 곳에서만
너는 팀원들이 끝날 때까지 기다리고 직접 구현하지 마. 공통 파일 변경 요청이 오면 그것만 처리해.   # 팀장이 손대면 충돌
```

⌨️ **팀이 일하는 동안**

```
Ctrl + T                                  # 선행 작업이 끝나면 잠긴 일이 풀리는지 관찰
↓ → be 고르고 Enter                        # be 화면: 지금 어떤 API 를 만드는지
지금 만든 API 를 curl 로 한 번씩 호출해서 응답이 api.md 예시와 같은지 보여줘   # be 에게 직접 검증 요청
팀원들이 끝날 때까지 기다려. 직접 구현하지 마   # 팀장이 직접 코드를 쓰기 시작하면
지금 어디까지 했어? 막힌 게 있어?           # 오래 멈춘 팀원 화면에서
```

⌨️ **터미널** (30분쯤 지나서 중간 확인)

```
git log --oneline                         # "be: 상품 목록 API", "shop: 카테고리 페이지" 처럼 팀원별 커밋이 쌓이는지
npm run dev                               # 브라우저로 지금까지 된 화면 눌러보기 → Ctrl + C
```

> 🧑‍🏫 **강사 해설**
>
> 60분 동안 보통 이렇게 진행됩니다. 처음 15~20분은 be 가 상품 목록·상세·검색 API 를 만들고, shop 은 api.md 예시로 상품 카드와 목록 화면 모양을 잡고, order 는 로그인·가입 화면 모양을 잡습니다. be 의 상품 API 가 완료되면 할 일 보드에서 shop 의 "목록·상세 연결"이 풀리고, 인증·장바구니 API 가 끝나면 order 의 "장바구니·주문 연결"이 풀립니다. 끝날 때쯤 `git log` 에는 팀원 이름이 붙은 커밋이 15~25개 쌓입니다.
>
> 검문소가 실제로 일하는 장면을 꼭 보여 주세요. 팀원이 빌드를 깨뜨린 채 완료하려다 거부당하고, 에러 메시지를 읽고 고친 뒤 다시 완료하는 장면이 거의 반드시 나옵니다. 사람이 아무 말도 안 했는데 버그가 커밋 전에 걸러진 것입니다. "버그 없이"의 나머지 절반이 이 자동 검사입니다.
>
> 흔한 사고는 셋입니다. 첫째, 팀장이 기다리지 않고 직접 화면을 만들기 시작합니다. 바로 멈추게 합니다. 둘째, order 가 장바구니 합계를 화면에서 직접 계산합니다. CLAUDE.md 의 "계산은 pricing.js 한 곳"을 어긴 것이므로, 서버가 준 합계를 그대로 보여 주게 고치라고 지시합니다. 셋째, 팀원이 api.md 와 다른 필드 이름을 쓰고 싶다고 팀장에게 물어옵니다. 이건 사고가 아니라 **정상**입니다. 계약 변경을 팀장이 받아서 api.md 와 `api.js` 를 한 번에 고치고 모든 팀원에게 알리는 게 맞는 흐름입니다.
>
> be 에게 curl 로 응답을 보여 달라고 직접 말 거는 연습은, "팀원이 완료라고 했지만 진짜로 계약대로인지"를 사람이 중간에 한 번 찔러 보는 습관을 들이기 위한 것입니다.

4-3. 1차 결과 확인 + 퇴근 (15분)

💬 **클로드 대화창**

```
할 일 보드에 끝나지 않은 일이 있으면 알려주고, 팀원 모두 퇴근시켜줘.   # 남은 일 점검 + 종료
```

⌨️ **터미널**

```
npm test && npm run build                 # 둘 다 통과해야 다음 블록으로
npm run dev                               # 홈 → 카테고리 → 상세 → 장바구니 → 로그인 → 주문 → 마이페이지 끝까지 눌러보기 → Ctrl + C
```

> 🧑‍🏫 **강사 해설**
>
> 이 시점에 학생 화면에는 이미 "쇼핑몰처럼 보이는" 사이트가 있습니다. 홈에 배너와 상품 카드가 나오고, 카테고리·검색·상세가 동작하고, 로그인해서 장바구니에 담고 주문하면 마이페이지에 주문이 보입니다.
>
> 하지만 학생이 직접 끝까지 눌러 보면 자잘한 문제가 몇 개는 나옵니다. 예를 들어 로그아웃 후에도 장바구니 숫자가 남아 있거나, 새로고침하면 로그인이 풀리거나, 모바일 폭에서 상품 카드가 넘칩니다. **각 팀원이 자기 파일 범위 안에서는 테스트를 통과했지만, 화면과 서버를 이어서 사람이 쓰는 흐름 전체는 아무도 검사하지 않았기 때문**입니다. 학생들에게 몇 개를 발견했는지 칠판에 적게 하세요. 이 숫자가 블록 5 의 동기가 됩니다.

🍚 점심 (60분)

블록 5

QA 팀원 — 사용자처럼 끝까지 눌러 보는 테스트 (40분)

목표 — 브라우저를 자동으로 움직이는 테스트(E2E)로 "사람이 쓰는 흐름 전체"를 검사합니다. 블록 4 의 검문소는 파일 단위만 봤기 때문에, 화면과 서버가 이어진 곳의 버그는 이걸로 잡아야 합니다. 테스트를 만드는 사람과 기능을 만든 사람을 다르게 두는 것이 핵심입니다.

5-1. QA 팀원 뽑기 (30분)

💬 **클로드 대화창**

```
agent team 으로 qa 팀원 1명을 뽑아줘. 모델은 Sonnet.   # 기능을 안 만든 새 눈
qa 의 파일 범위는 e2e/** 와 playwright.config.js 만이야. 앱 코드는 고치지 말고, 버그를 찾으면 파일:줄과 재현 방법을 너(팀장)에게 보고해.   # 테스트 담당은 고치지 않고 보고만
할 일: Playwright 를 설치하고(npx playwright install chromium) 아래 흐름을 e2e 테스트로 만들어.   # 실제 브라우저로 클릭
1) 홈 → 카테고리 → 정렬을 낮은가격순으로 → 첫 상품 가격이 가장 낮은지   # 정렬이 진짜 되는지
2) 검색창에 두 글자 → 자동완성 → 상품 상세 → 옵션 선택 → 수량 2 → 장바구니 → 합계가 할인가×2 인지   # 가격 계산이 화면 끝까지 맞는지
3) 비로그인으로 주문 → 로그인 화면으로 → 로그인 → 다시 주문서로 돌아오는지   # 비로그인 흐름
4) 주문 → 주문 완료 → 마이페이지에 그 주문이 있고, 재고가 줄었는지   # 주문이 끝까지 저장되는지
5) 새로고침해도 로그인과 장바구니가 유지되는지, 로그아웃하면 장바구니 숫자가 사라지는지   # 새로고침·로그아웃
6) 모바일 크기(390×844)에서 홈과 상세에 가로 스크롤이 생기지 않는지   # 반응형
npm run e2e 로 돌아가게 package.json 스크립트 변경은 너(팀장)가 해.   # 공통 파일은 팀장
```

⌨️ **터미널**

```
npm run e2e                          # 브라우저 테스트 실행 → 통과/실패 개수 확인
npx playwright show-report           # 실패한 테스트의 화면 캡처·단계별 기록 보기
```

👩‍🏫 강사 해설

결과로 `e2e/` 폴더에 테스트 파일들이 생기고, `npm run e2e` 를 돌리면 **몇 개는 실패**하는 게 정상입니다. 블록 4 끝에 학생들이 손으로 찾은 문제(로그아웃 후 장바구니 숫자, 새로고침 시 로그인 풀림, 모바일 가로 스크롤 등)가 대부분 여기서 자동으로 걸립니다. 거기에 사람이 못 찾은 것(정렬이 두 번째 페이지부터 안 맞음, 재고가 줄지 않음 등)이 1~2개 더 나오면 QA 가 제 몫을 한 것입니다.

qa 에게 앱 코드를 못 고치게 한 이유는 1차 실습의 test-runner 와 같습니다. 테스트를 쓰는 사람이 직접 고치기 시작하면 "테스트를 통과하도록 테스트를 느슨하게" 바꾸는 유혹이 생깁니다. 찾는 사람과 고치는 사람을 나누면 테스트가 기준으로 남습니다.

`show-report` 화면을 학생들에게 보여 주세요. 실패한 순간의 화면 캡처와 어떤 버튼을 눌렀는지가 단계별로 남아서, 버그 위치를 찾기가 훨씬 쉽습니다.

Playwright 설치(`npx playwright install chromium`)는 브라우저를 내려받느라 수백 MB 가 들고 몇 분 걸립니다. 교실 네트워크가 느리면 이 단계 시작 전에 다 같이 미리 설치해 두세요.

5-2. 실패 목록 정리 (10분)

💬 **클로드 대화창**

```
qa 의 보고와 npm run e2e 실패를 합쳐서 docs/bugs.md 에 정리해줘. 열: 번호 ¦ 증상 ¦ 원인 파일:줄 ¦ 담당 팀원(be/shop/order/팀장) ¦ 재현 e2e 테스트 이름.   # 오후 수리의 할 일 목록
qa 는 퇴근시키지 마. 블록 7 에서 다시 돌릴 거야.   # 같은 qa 를 재사용
```

👩‍🏫 강사 해설

`docs/bugs.md` 에 담당 팀원 열이 있으면 정답입니다. 담당은 블록 4 의 파일 소유표로 정해집니다. 예를 들어 "로그아웃 후 장바구니 숫자가 남음"은 `client/src/store/` 문제라 order, "재고가 안 줄어듦"은 서버 주문 처리라 be 입니다. 버그 목록을 파일 소유표로 나누면 블록 7 에서 다시 팀원끼리 겹치지 않게 고칠 수 있습니다.

qa 를 살려 두는 이유는, 블록 7 에서 고친 뒤 같은 qa 에게 다시 돌리게 하면 자기가 쓴 테스트를 이미 알고 있어서 빠르기 때문입니다. 다만 쉬는 동안에도 말을 걸면 사용량을 쓰므로, 블록 6 동안은 qa 에게 말을 걸지 않습니다.

블록 6

리뷰 팀 — 보안·성능·접근성 털기 (40분)

목표 — 기능이 "돌아가는 것"과 "안전하고 빠른 것"은 다릅니다. 관점이 다른 리뷰어 3명이 각자 맡은 파일을 깊게 보고, 직접 실행해서 증명한 문제만 남깁니다. 1차 실습에서 확인한 것처럼 팀 리뷰의 강점은 개수보다 **확인의 깊이**입니다.

6-1. 리뷰 팀 (30분)

💬 **클로드 대화창**

```
agent team 으로 리뷰 팀 3명을 뽑아줘. 모델은 Sonnet. 코드는 고치지 마.   # 리뷰는 읽고 증명만
- sec: server/src/** 와 client/src/api.js 의 보안. 가격 조작(화면이 보낸 가격을 믿는지), 남의 주문·장바구니 보기, 비밀번호 저장, 토큰, 입력 검증   # 쇼핑몰에서 돈과 개인정보가 걸린 곳
- perf: 서버 응답 속도와 프론트 번들 크기. 상품 목록 API 가 페이지 단위로만 주는지, 빌드 결과 크기, 이미지·폰트 낭비   # 느리면 쇼핑몰은 끝
- a11y: client 화면의 접근성과 모바일. 키보드로 주문까지 가능한지, 버튼 이름, 색 대비, 390px 폭   # 누구나 쓸 수 있는지
각자 재현한 것만 "재현", 코드로만 본 건 "추측" 으로 표시. 서로 결과를 보고 과장된 건 반박해서 걸러.   # 1차 실습의 교훈 그대로
결과를 합쳐서 docs/review.md 에 심각도 순으로 써줘.
```

> 👩‍🏫 **강사 해설**
>
> 쇼핑몰 리뷰에서 가장 중요한 항목은 **가격 조작**입니다. sec 가 장바구니나 주문 요청에 `price` 필드를 1 로 바꿔 보내 보고, 서버가 그 값을 무시하고 자기 데이터로 다시 계산하는지 확인합니다. 블록 1 의 critic 이 "가격은 서버가 계산한다"를 계약에 넣었고 블록 2 에서 CLAUDE.md 에 못 박았다면, 여기서는 "재현해 봤지만 막혀 있음"이 나와야 합니다. 이게 나오면 설계 단계에서 막은 버그가 실제로 막혔다는 증거이니 꼭 짚어 주세요. 만약 뚫리면 그건 "계약을 썼는데 구현이 안 지킨" 경우라서, 리뷰 팀이 왜 필요한지 보여 주는 좋은 예가 됩니다.
>
> 다른 단골 항목은 이렇습니다. 주문 id 만 바꾸면 남의 주문이 보이는지(sec), 상품 목록 API 가 60개를 한 번에 다 주는지(perf), 상품 카드가 `div` 라서 키보드 Tab 으로 갈 수 없는지(a11y), 할인율 빨간 글씨의 색 대비가 부족한지(a11y).
>
> a11y 를 리뷰어로 넣은 이유도 말해 주세요. 쇼핑몰은 누구나 쓰는 서비스라 키보드나 화면 낭독기로도 주문까지 갈 수 있어야 합니다. 그리고 버튼에 이름이 제대로 붙어 있으면 블록 5 의 E2E 테스트도 버튼을 이름으로 찾을 수 있어서 테스트가 덜 깨집니다. 접근성과 테스트하기 좋은 구조는 같은 방향입니다.

6-2. 리뷰 팀 퇴근 + 저장 (10분)

💬 **클로드 대화창**

```
리뷰 팀 3명만 퇴근시켜줘. qa 는 남겨.   # 이름으로 골라서 퇴근
docs/review.md 의 "재현" 항목을 docs/bugs.md 에 담당 팀원과 함께 추가해줘.   # 수리 목록 하나로 합치기
```

⌨️ **터미널**

```
git add docs && git commit -m "docs: E2E 실패 + 리뷰 결과 → 수리 목록"   # 수리 전 세이브
```

> 👩‍🏫 **강사 해설**
>
> "추측" 항목은 수리 목록에 넣지 않는 이유를 설명해 주세요. 증명되지 않은 문제까지 고치면 멀쩡한 코드를 건드리다 새 버그를 만들 수 있습니다. 추측 항목은 따로 남겨 두고, 시간이 남으면 블록 8 에서 확인합니다.

☕ 휴식 (10분)

블록 7

수리 + 다듬기 팀 — "개쩌는" 마무리 (50분)

목표 — 버그 목록을 파일 주인별로 나눠 고치고, 남는 시간에 쇼핑몰다운 마감(배너 애니메이션, 스켈레톤 로딩, 빈 화면 안내, 토스트 알림)을 더합니다. 고칠 때마다 qa 의 E2E 를 다시 돌려서 고치다 다른 걸 깨뜨리지 않았는지 확인합니다.

7-1. 수리 팀 (35분)

💬 **클로드 대화창**

```
docs/bugs.md 를 담당별로 나눠서 be, shop, order 팀원을 다시 뽑아줘(Sonnet). 블록 4 와 같은 파일 소유표.   # 같은 소유표 = 같은 규칙
각자 자기 버그를 고칠 때마다 그 버그를 잡는 테스트를 먼저 추가하고(서버는 server/test, 화면은 qa 에게 e2e 추가 요청), 고친 뒤 통과시켜.   # 재발 방지 테스트 먼저
자기 버그가 끝나면 남은 시간에 다듬기: shop 은 홈 배너 자동 슬라이드·스켈레톤 로딩·랭킹 순위 뱃지, order 는 장바구니 담기 토스트·빈 장바구니 안내·주문 완료 화면 꾸미기.
# 쇼핑몰다운 마감
모든 팀원 완료 후 qa 에게 npm run e2e 전체를 다시 돌리게 해줘.   # 마지막에 전체 재검사
```

⌨️ **팀이 일하는 동안**

```
Ctrl + T                                # 버그별 할 일이 완료로 바뀌는지
↓ → qa 고르고 Enter                      # qa 화면: 팀원들의 e2e 추가 요청을 받고 있는지
```

> 🧑‍🏫 **강사 해설**
>
> "고치기 전에 그 버그를 잡는 테스트부터"가 이 블록의 원칙입니다. 테스트가 먼저 실패하고, 고친 뒤 통과하면 그 버그는 다시 생겨도 검문소가 막습니다. 이 순서 없이 고치기만 하면 다음 수정 때 같은 버그가 조용히 돌아옵니다.
>
> 이 블록에서 팀원끼리 메시지를 주고받는 장면이 자연스럽게 나옵니다. order 가 qa 에게 "로그아웃 후 장바구니 숫자 테스트 추가해 줘"라고 요청하고, qa 가 테스트를 추가하면 order 가 고칩니다. 심부름꾼은 서로 대화하지 못하므로 이런 협업은 팀에서만 됩니다.
>
> 다듬기 항목을 버그 수리 **뒤에** 두는 이유도 설명해 주세요. 화면을 화려하게 만드는 일은 끝이 없어서, 먼저 시작하면 버그 수리가 밀립니다. 그리고 다듬기도 검문소와 E2E 를 통과해야 하므로, 화려해지는 동안 기능이 깨지지 않습니다.
>
> 마지막 `npm run e2e` 전체가 모두 통과하면 정답입니다. 1~2개가 남으면 블록 8 에서 같이 봅니다.

7-2. 전원 퇴근 + 저장 (15분)

💬 **클로드 대화창**

```
팀원 모두 퇴근시켜줘. qa 도 포함.   # 오늘 팀은 여기까지
```

⌨️ **터미널**

```
npm test && npm run build && npm run e2e     # 세 가지 모두 통과해야
git add . && git commit -m "fix: 수리 + 다듬기 완료"   # 최종 세이브
git log --oneline ¦ wc -l                    # 오늘 쌓인 커밋 개수 (보통 40~70개)
```

> 🧑‍🏫 **강사 해설**
>
> 커밋 개수를 세게 하는 이유는, 학생이 손으로 쳤다면 며칠 걸렸을 작업이 하루에 쌓였다는 걸 숫자로 보여 주기 위해서입니다. 동시에 "이 커밋들을 내가 다 읽었는가?"도 물어 주세요. 답은 대부분 "아니요"입니다. 그래서 블록 8 의 검증이 필요합니다. 내가 다 읽지 못한 코드를 믿으려면, 처음 받는 사람의 자리에서 돌려 보는 확인이 있어야 합니다.

블록 8

최종 검증 — 처음 받는 사람처럼 (40분)

목표 — 지금 폴더에서 되는 건 "내 컴퓨터에서만" 될 수도 있습니다. 완전히 새 폴더에 복사해서 설치부터 다시 해 보고, 사람이 직접 처음부터 끝까지 써 봅니다. 팀이 만든 결과물을 믿을 수 있는지 마지막으로 확인하는 단계입니다.

8-1. 새 폴더에서 처음부터 (20분)

⌨️ **터미널**

```
cd ~/projects
git clone haru-market haru-check          # 커밋된 것만 새 폴더로 복사 (커밋 안 한 파일은 안 따라옴)
cd haru-check
npm install                               # 처음 받은 사람처럼 설치부터
npm test && npm run build && npm run e2e  # 세 가지가 새 폴더에서도 통과하는지
npm run dev                               # 직접 써 보기 → Ctrl + C
```

> 🧑‍🏫 **강사 해설**
>
> 새 폴더에서도 세 가지가 모두 통과하면 정답입니다. 여기서 자주 걸리는 건 세 종류입니다. 팀원이 라이브러리를 설치만 하고 `package.json` 에 저장하지 않은 경우(새 폴더에서 `Cannot find module`), 데이터 파일이 `.gitignore` 에 걸려 커밋되지 않은 경우(상품이 0개), 팀원이 만든 파일을 아무도 커밋하지 않은 경우입니다. 셋 다 "원래 폴더에서는 되니까 아무도 몰랐던" 버그입니다.
>
> 걸린 게 있으면 원래 폴더(`haru-market`)로 돌아가 클로드 혼자에게 고치게 하고, 커밋한 뒤 `haru-check` 에서 `git pull` 로 다시 확인합니다. 이 정도 마무리는 팀이 아니라 **혼자**가 맞습니다. 순서대로 이어지는 작은 수정이라 팀원이 서로 기다리기만 하기 때문입니다.

8-2. 사람이 직접 써 보기 체크 (20분)

⌨️ **브라우저에서 직접** (짝꿍과 바꿔서 서로의 쇼핑몰 써 보기)

```
가입 → 로그인 → 홈 배너·타임특가 확인 → 카테고리 정렬·가격 필터 → 검색 자동완성   # 둘러보기
상세에서 품절 옵션 고르기 → 장바구니 담기 → 수량 바꾸기 → 합계 확인              # 가격 계산
주문 → 주문 완료 → 마이페이지 주문내역 → 같은 상품 재고 줄었는지                 # 주문 흐름
로그아웃 → 장바구니 숫자 → 다시 로그인 → 찜·최근 본 상품                        # 계정 상태
브라우저 창을 휴대폰 폭으로 줄여서 전부 다시                                    # 모바일
```

> 🧑‍🏫 **강사 해설**
>
> 짝꿍의 쇼핑몰을 써 보게 하는 이유는, 내가 만든 건 내가 아는 길로만 누르기 때문입니다. 남이 쓰면 예상 못 한 순서로 누르다가 버그를 찾습니다. 찾은 게 있으면 짝꿍에게 알려 주고, 그게 E2E 테스트에 없던 흐름이라면 "테스트가 놓친 곳"으로 칠판에 적습니다. 이 목록은 블록 9 회고 때 "다음에 처음부터 테스트에 넣을 흐름"이 됩니다.
>
> 여기서 학생들이 거의 버그를 못 찾는다면, 오늘 순서(계약 → 뼈대 → 파일 주인 → 검문소 → E2E → 리뷰 → 수리)가 제대로 작동한 것입니다. "한 방에 버그 없이"는 운이 아니라 이 순서의 결과라는 점을 강조하며 마무리합니다.

블록 9

발표 & 회고 (30분)

목표 — 학생 5~6명이 2분씩 자기 하루마켓을 시연하고, 어디서 팀이 이득이었고 어디서 혼자가 맞았는지 정리합니다.

⌨️ **발표 순서** (한 사람 2분)

```
1) 내 쇼핑몰 시연: 홈 → 검색 → 상세 → 장바구니 → 주문 → 마이페이지   # 결과물
2) git log --oneline 개수와 /usage 로 본 오늘 사용량                  # 비용
3) 팀이 엇나간 순간 하나 + 내가 어떻게 바로잡았나                      # 운영 경험
4) 검문소나 E2E 가 막아 준 버그 하나                                  # 자동 검사의 가치
```

👩‍🏫 강사 해설

오늘 하루를 이렇게 묶어서 정리해 주세요.

단계	누가	왜 그 방식이었나
설계(블록 1)	🏢 팀 4명	관점이 독립적이고 파일을 안 고침. critic 의 반박으로 가격 조작·소수점 버그를 문서 단계에서 막음
계약 확정(블록 2)	🧑 나	계약은 사람이 서명. CLAUDE.md 로 모든 팀원에게 같은 규칙
뼈대(블록 3)	🧑 팀장 혼자	package.json·라우터 같은 공통 파일은 여럿이 만들면 반드시 덮어씀
구현(블록 4)	🏢 팀 3명	파일이 겹치지 않게 나눌 수 있어서 동시에 진행 → 가장 큰 속도
E2E(블록 5)	🏢 qa 1명	만든 사람과 검사하는 사람을 분리
리뷰(블록 6)	🏢 팀 3명	관점별로 깊게, 재현으로 증명
수리(블록 7)	🏢 팀 + qa	버그 목록을 파일 주인별로 나눔. 팀원끼리 테스트 요청 협업
최종 검증(블록 8)	🧑 혼자	순서대로 이어지는 작은 수정은 팀이 오히려 느림

"한 방에 버그 없이"의 정체는 결국 세 가지입니다. **모두가 같은 계약서를 보고 만든다**(블록 1·2), **공통 파일은 한 명만, 나머지는 파일 주인이 따로 있다**(블록 3·4), **사람 대신 기계가 매번 검사한다**(검문소·E2E). 이 셋이 없으면 팀은 혼자보다 버그를 더 많이 만듭니다. 이 셋이 있으면 팀은 혼자보다 몇 배 빠르면서도 덜 틀립니다.

사용량 이야기도 꼭 해 주세요. 오늘은 팀원을 블록마다 뽑고 바로 퇴근시켰습니다. 팀원은 살아 있는 동안 말을 걸 때마다 깨어나 자기 대화 전체를 다시 읽기 때문에, 쓰고 나면 바로 퇴근시키는 습관이 사용량을 가장 크게 아낍니다.